MUST KNOW - BSV3

THOMAS HASLWANTER

Sept-2025

1 MATH BASICS

1.1 Lines, Scalar- and Vector-Product

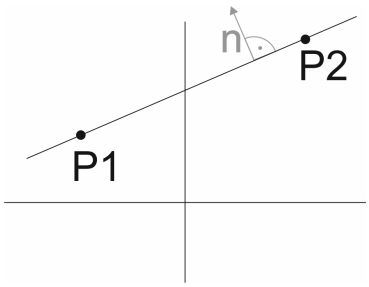


Figure 1: The vector perpendicular to `[a , b]` is `[b, -a]`.

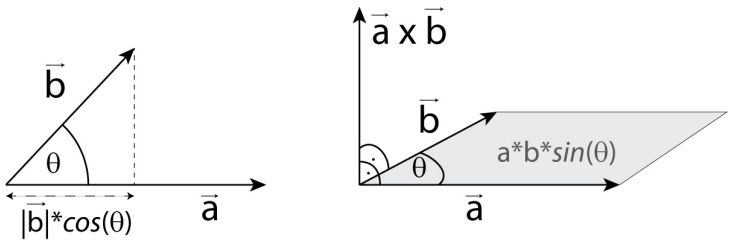


Figure 2: Scalar product and vector product

1.2 Complex Numbers and Trigonometry

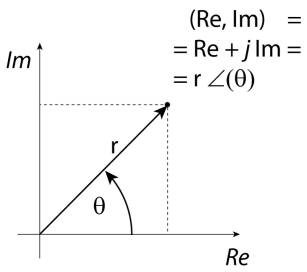
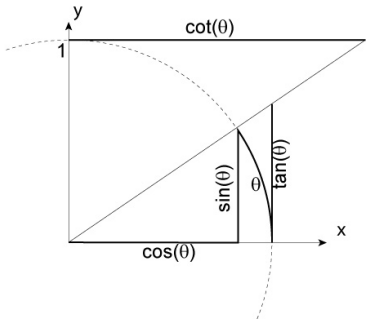


Figure 3: $r = \sqrt{Re^2 + Im^2}$, $\theta = \arctan(Im/Re)$

* Figures from "Hands-on Signal Analysis with Python", Haslwanter, Springer 2021



$$e^{j\omega t} = \cos \omega t + j \sin \omega t \tag{1}$$

1.3 Nyquist Theorem

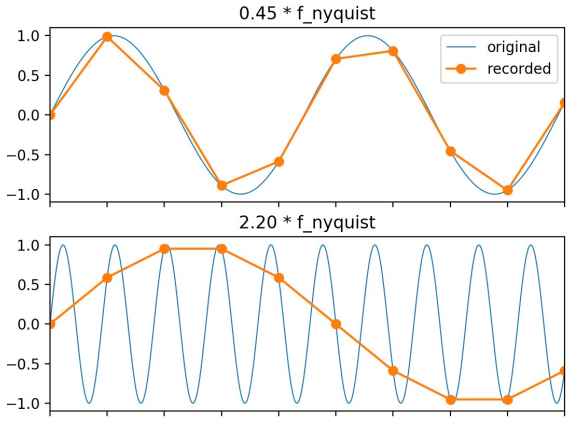


Figure 4: The *Nyquist frequency* is half the sample rate. Signals above the Nyquist frequency can not be recorded correctly.

1.4 Matrix Multiplication

In addition, you are required to know how to work with vectors and matrices.

2 PYTHON

- Tuples, lists, np.array-s, dictionaries, and Pandas DataFrames
- Basic manipulation of vectors and arrays (e.g. multiplication of matrix with data, point-wise multiplication vs matrix multiplication)
- "Slice" out selected data from an array (e.g.: the last 10 values from column 2)
- Formatted text output (e.g.: 'The value of pi is 3.14')
- Function definitions, default parameters, return of multiple elements

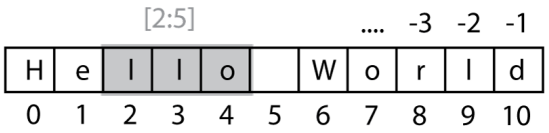


Figure 5: Attention: the last element is not included! And note that you can count from the end, using negative positions.

Important Commands

```
import ...
np.mean(x, axis=[0/1])
dict.keys(), dict.values()
data_frame.values => np.array
data_rotated = (rot_mat @ data.T).T     # for data in column form
print(f'The value of pi is {np.pi:.2f}')

def func_name(in_data=default_val):
  """ Comment """
  ...

  return (a, b)
```

3 FILTERING

IMPULSE Single `1` in a sequence of `0`'s

FIR Finite Impulse Response

IIR Infinite Impulse Response

3.1 FIR - Finite Impulse Response

Implementation: `scipy.signal.lfilter(b, 1, x)`

MOVING AVERAGE b-coefficients

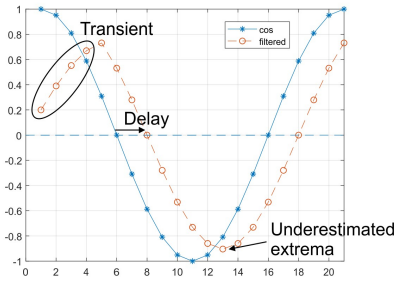


Figure 6: Effects of moving-average filter on a cosine-signal.

SAVITZKY-GOLAY Parameters

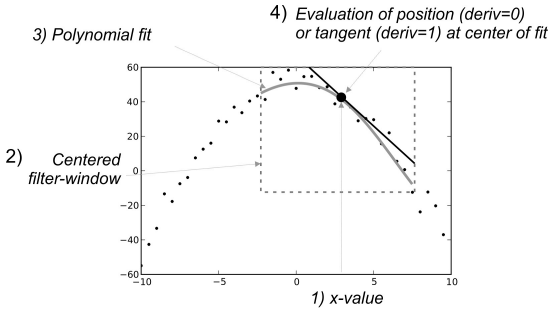


Figure 7: Savitzky-Golay filter

2-POINT DIFFERENTIATION b-coefficients

3-POINT DIFFERENTIATION b-coefficients

3.2 IIR - Infinite Impulse Response

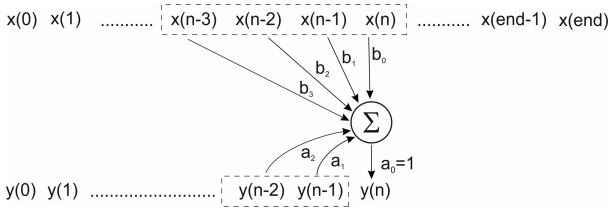


Figure 8: IIR-Filters. For FIR-filters: `a=[1]`

Implementation: `scipy.signal.lfilter(b, a, x)`

BUTTERWORTH High-pass and low-pass with one parameter, band-pass and notch-filter with two parameters. How do you obtain the `[a, b]`-coefficients? How do you enter the upper/lower bandwidth(s)?

EXPONENTIAL AVERAGING FILTER How do you get the `[a, b]`-coefficients?

Implementation: `scipy.signal.lfilter(b, a, x)`

3.3 Non-linear filters

MEDIAN-FILTER When the signal has extreme outlier

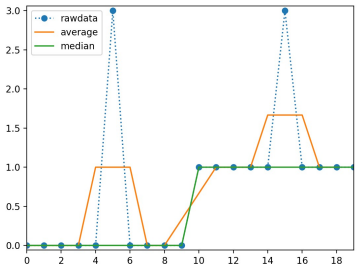
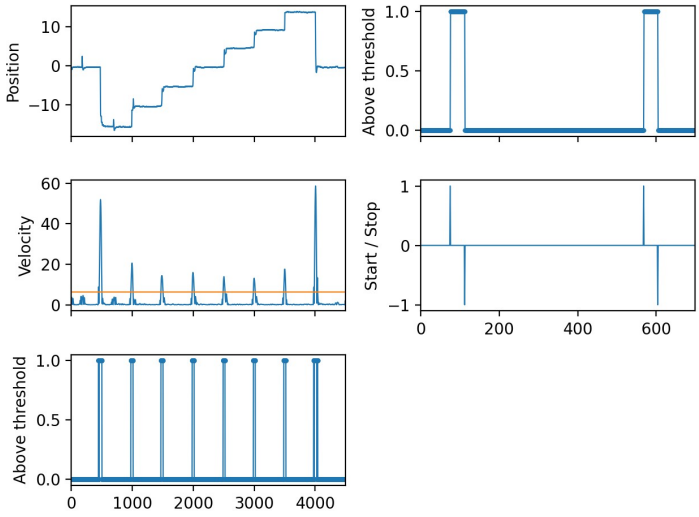
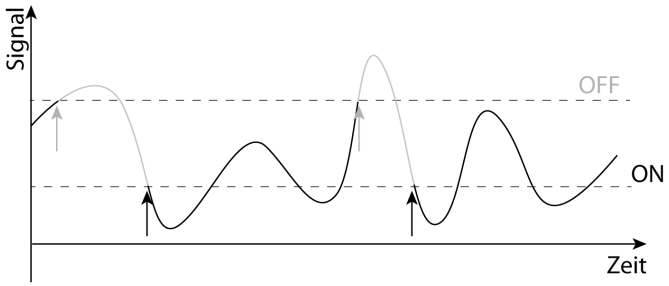


Figure 9

4 EVENTS

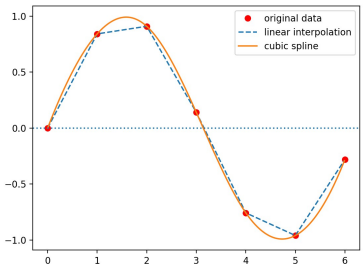
4.1 Event Detection





Implementation: `np.where(np.diff(x ≶ threshold)==±1)[0]`

4.2 Interpolation



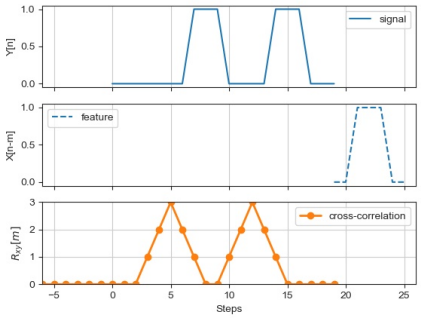

```
from scipy.interpolate import CubicSpline
cs = CubicSpline(x, y)
x_int = ...
y_int = cs(x_int)
```

4.3 Cross Correlation

- How to calculate the cross-correlation of e.g. `a = [3, 4, -1]` with `b = [5, 2, -1, 1]`
- Cross correlation ↔ convolution ↔ FIR-filter

Normalization:

- Offset
- Amplitude: $\mathrm{sig}_{normalized} = \frac{\mathrm{sig}_{raw}}{\sqrt{\max(\mathrm{autocorr}(\mathrm{sig}_{raw}))}}$
- Duration



5 STATISTICS

Box plots use quartiles to describe asymmetrical sample data, or sample data with distinct outliers.

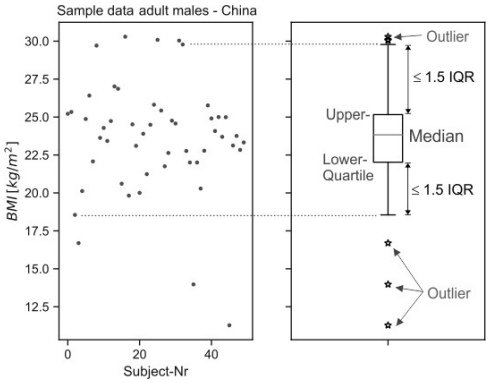


Figure 10: Left: Scatter plot **Right:** Corresponding box plot. The difference between the upper and lower quartile is called the *Inter-Quartile-Range (IQR)*. Error-bars commonly indicate the most extreme values inside *1.5*IQR* outside the upper/lower quartile.

Based on our data, the best estimate we can get for the expected standard deviation of the *population*(!) is

$$s = \sqrt{\frac{\sum (w_i - \bar{w})^2}{n-1}} \tag{2}$$

This is called the *sample standard deviation* (because it is our best guess based on the sampled data). The mathematical definition of the normal distribution is:

$$f_{\mu,\sigma}(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2} \, , -\infty < x < \infty \tag{3}$$

$f_{\mu,\sigma}$ is called the *probability density function (PDF)* of the normal distribution. The integral of the PDF between *a* and *b* is the probability that the next sample value will lie in that interval.

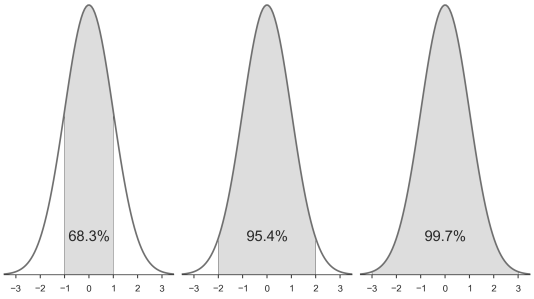


Figure 11: Normal Distribution: the unit on the horizontal axis is 1 standard deviation (SD). Data points lying outside ±2 SD are called "significantly different"; data points outside ±3 SD are called "highly significantly different".

The standard deviation tells us about the variability of the data, and the standard error gives us the uncertainty of a parameter. For the mean value of normally distributed data, we have:

$$\mathrm{SEM} = \frac{\sigma}{\sqrt{n}} \tag{4}$$

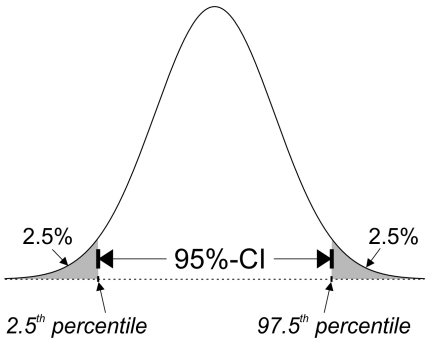


Figure 12: Values outside the 95% confidence interval (CI) are called *significantly different*, values outside the 99.9%-CI are called *highly significantly different*.

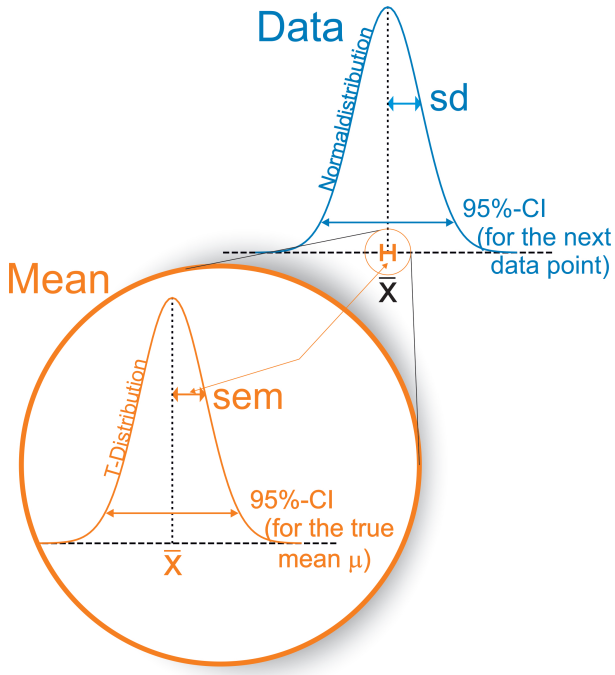


Figure 13: While the normal distribution describes the variability of the data, the T-distribution describes the variability of the mean value.

T-TESTS

- one group vs value
- independent groups
- paired T-test

Interpretation of p-value for T-tests

p is the probability to obtain a value as extreme or more extreme than the observed one, if the null hypothesis is true.

6 PARAMETER FITS

r Correlation coefficient: how well data can be described with a line. $-1 < r < 1$
$r \propto \sum_i \Delta x_i * \Delta y_i$

r^2 Determination coefficient: percentage of the variability in y that is explained by the linear fit

6.1 Linear Model with Intercept

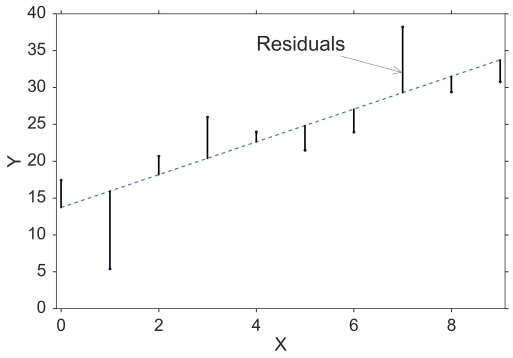


Figure 14: For a *best fit*, the sum square of the **residuals** has to be minimized.

The general linear model with intercept term is given by

$$y_k \approx \left(\beta_0 + \sum_{i=1}^{p} \beta_i x_{ki} \right)$$

or in matrix form

$$\mathbf{y} = \mathbf{X} \cdot \boldsymbol{\beta} \tag{5}$$

with the following matrix and vector definitions:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}, \ \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}, \ \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \tag{6}$$

The matrix **X** is sometimes called *design matrix*, and equations that can be written as above (where the parameters enter the equation only linearly) are called `linear regressions`.

The most important feature of this type of model is that the parameters β_i that we are looking for enter the model only linearly.

6.2 Line-Fit

For a line-fit, Eq. 6 is reduced to

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \dots \end{bmatrix} = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ x_3 & 1 \\ \vdots & \vdots \end{bmatrix} \cdot \begin{bmatrix} m \\ b \end{bmatrix}$$

```
...

# Calculate the best-fit line
M = np.column_stack( [x, np.ones_like(x)] )
p_estimator = np.linalg.pinv(M) @ y

slope = p_estimator[0]        # "m"
y_intercept = p_estimator[1]  # "c"
```

Note that the sequence of columns in the design matrix determine the sequence of parameters in the parameter vector.

CONFIDENCE-INTERVALS FOR FITTED PARAMETERS With the design matrix `X` and the result vector `y`, the package `pingouin` can find the best fit parameters as well as the corresponding confidence-intervals:

```
import pingouin as pg
results = pg.linear_regression(X,y)
ci_lower = results['CI[2.5%]']
ci_upper = results['CI[97.5%]']
```

6.3 Polynomial-Fit

We can use the same approach to fit a polynomial curve to the data. For example, for a quadratic relationship between x and y we have

$$y = a * x^2 + b * x + c \tag{7}$$

Written in matrix form this gives

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \dots \end{bmatrix} = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \\ \vdots & \vdots & \vdots \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \end{bmatrix} \tag{8}$$

With this we have brought the problem into the form $\mathbf{y} \approx \mathbf{X} \cdot \boldsymbol{\beta}$, and we can solve it in the same way as above. Or in other words:

> We can use a linear model to fit a quadratic (or higher order) polynomial, because the model parameters enter our equation only in a linear way.

7 FOURIER TRANSFORMATION

The Fourier Transformation (Fig. 15) is given by

$$X_n = \sum_{\tau=0}^{N-1} x_\tau * e^{-j2\pi \frac{n \cdot \tau}{N}} \quad \text{with } n = 0, \dots, N-1\,. \tag{9}$$

The nth Fourier coefficient provides amplitude and phase for the nth frequency, with

$$amplitude = |X_n| \tag{10}$$

$$phase = atan(\frac{X_n.imag}{X_n.real}) \tag{11}$$

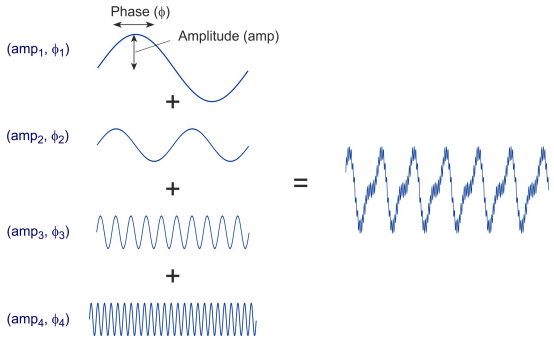


Figure 15: Basic idea behind the Fourier Series

For real-valued signals the Fourier components obey

$$X(f) = X(-f)^* \tag{12}$$

where "*" indicates the complex conjugate. In words, for real-valued signals the Fourier transform needs to include positive and negative frequency components of the same magnitude, so that the imaginary contributions of the two can cancel out → the powerspectrum (= $|FFT^2|$) is symmetrical about the Nyquist frequency.

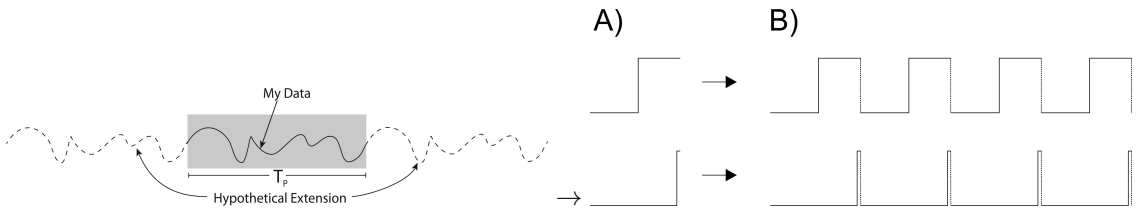


Figure 16: Left: The *Fourier Series* assumes that the measured signal is one repetition of a repeating sequence. **Right:** The consequence: the FFT of a step-signal depends on *where* the step appears.

- The first Fourier component corresponds to $e^{j0} = 1$, and is therefore proportional to the offset of the signal.
- The lowest oscillating frequency is $\frac{1}{N \cdot T_s}$ [Hz] , i.e. $\frac{1}{signalduration}$.
- For real valued data, the highest independent frequency is half the sampling frequency, $\frac{1}{2T_s}$ (Nyquist-Shannon theorem).
- For real valued data, the upper half of the Fourier coefficients have to be the complex conjugate of the lower half.

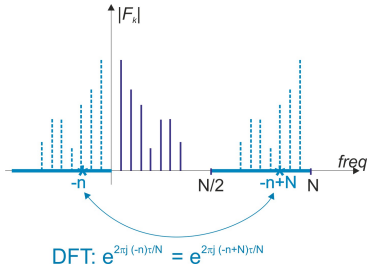


Figure 17: *Negative frequencies* correspond to frequencies above the Nyquist frequency.